

An-Najah National University

Computer Engineering Department

VLSI Design Verification

Assignment #11: Final assignment

Functional Coverage

Constraint-random testing (CRT) is very dominant in modern test bench and verification approaches. For CRT to be effective, it is usually combined with functional coverage to clearly quantify the how good is the CRT stimulus as a first step. Advanced functional coverage goes beyond identifying how some variables are driven by the TB or the DUT, but also identifying complex stimulus/output combinations and how well the TB drives these complex scenarios.

In this exercise, the goal is to experiment with functional coverage to find bugs.

Assignment weight: 5%

Instructions:

1. A black boxed DUT is provided which is similar to the design we used in the previous exercise, except the fact that it has an accumulate feature (either add or multiply). The **acc** bit enables the accumulate feature and the **func** bit decides whether it is an accumulate add write or accumulate multiply write:

```
module reg_ctrl
# (
    parameter ADDR_WIDTH = 8,
    parameter DATA_WIDTH = 24,
    parameter DEPTH      = 256,
    parameter RESET_VAL  = 24'h123456    // Adjusted reset value
)
(
    input clk,
    input rstn,
    input [ADDR_WIDTH-1:0] addr,
    input sel,
    input wr,
    input acc,    // accumulate
    input func,   // function
    input [DATA_WIDTH-1:0] wdata,
    output reg [DATA_WIDTH-1:0] rdata,
    output reg ready
);
```

Steps to run the black-boxed design:

- a. `tar -xvf vcs_blackbox_deliver.tar`
- b. `cd vcs_blackbox_deliver`
then prepare you TB in file called tb.sv where the top-level testbench module is called tb
- c. `vlog tb.sv -sverilog`
compiles the TB file(s) into the same library
can have multiple files. Just repeat this step for each file
- d. `vcs tb` # Should produce the simv binary/executable tb.

2. Update the TB to include the following features:
 - a. Make sure the TB has a transaction object (randomized) for driving the values to the DUT, Generator class connected to the Driver, Monitor and Checker classes.
 - b. Coverage: a special component/class where the functional coverage covergroup are defined and instantiated and transactions are monitored for updating the coverage. Could be connected to the monitor using a mailbox.
3. Define the following cover points (inside the coverage component/class):
 - a. cp1: tr.addr (18 bins?)
 - i. bins for only 1 bit set (e.g. 1, 2, 4, 8, 16, ...)
 - ii. bins for min, min+1, max, max-1
 - iii. bins for alternating bits set, e.g. 0b01010101, 0b10101010
 - iv. bins for middle of memory +-1, e.g. 0x7e, 0x7f, 0x80, 0x81
 - b. cp2: tr.wr
 - c. cp3: tr.wdata (sampled when tr.wr = 1)
 - i. bins similar to address bins (expanded to 24 bits)
 - ii. bins for only one byte set (0000ff, 00ff00, ff0000)
 - d. cp4: tr.acc
 - e. cp5: tr.func
 - f. cp6: tr.rdata (sampled when tr.wr = 0)
4. Define cross coverage items that you find suitable: e.g.
cp7: cross: cp1, cp2, cp3, cp4, cp5 cover points
5. Have the randomized TC have three phases:
 - a. After reset phase: send a large number of read transaction (to random addresses) to check the reset value.
 - b. Main phase: send a large number of randomized transactions.
 - c. Final phase: send a large number of read transactions to check the final value of the memory.
6. Always check the value of the ready signal is as expected (using an assertion?).
7. Try to achieve 100% functional coverage using pseudo random tests only.
8. Verify that the DUT works as expected: try to avoid driving specific values to the DUT (always use randomized objects for driving).

Submission:

Submit exactly 4 files and 4 files only (nothing else):

1. Complete SV Code in a single SV file (.sv)
2. Log file output from the terminal (.txt)
3. Functional Coverage screenshot from the simulator (.png)
4. 1-2 page report explaining the bugs you found (.pdf)

NO compressed folders please. Just upload 4 single files. Anything else would be penalized.

Good Luck